# Moderated Learning Management System: Final Report

CS4433 Software Engineering · Final Project · Mafeka M.M. · Student number 202321161

Repository (source code, diagrams and appendices): https://github.com/mamakhooamafeka/CS4433-software-engineering

**Abstract.** This report summarises the development of an offline Learning Management System for a secondary school, built in C++17 with SQLite through the eight phases of the Waterfall model, from planning to the maintenance release v1.1.0. The system supports four roles, applies late penalties and extensions, and lets Heads of Department moderate marks before learners see them, with every mark change audited. It meets all 23 functional requirements; 129 automated tests pass on Windows and Linux with 95.2% line coverage of the business logic. Diagrams D1 to D12 and Appendices A to H are kept in the repository and linked where they are used.

**Contents.** 1 Introduction · 2 Project management · 3 Requirements · 4 Design · 5 Implementation · 6 Verification and validation · 7 Configuration management and QA · 8 Deployment · 9 Maintenance · 10 Professional, legal and ethical issues · 11 Conclusion · References · Appendices A to H (index)

## 1 Introduction

The school needs to manage courses, assignments and assessment for students, lecturers, Heads of Department (HoDs) and an administrator, on old computers with unreliable internet. Its stakeholders disagreed about when marks are released, how late work is penalised and how simple the system must be. The project followed the Waterfall model with three supervisor gates, so each phase produced a baselined artefact that the next phase builds on and that the traceability matrix (App. A) links together.

## 2 Project management

The feasibility study assessed technical, operational, economic, legal and schedule feasibility and recommended a console C++ application with an embedded SQLite database, because it works offline on old hardware at no licence cost; cloud platforms and self-hosted Moodle were rejected for needing internet, a server or a workflow without moderation before release. The work breakdown structure (D1) has 49 work packages totalling 210 hours plus a 10-hour reserve. The Gantt chart (D2) schedules them over 49 days with finish-to-start dependencies, three gates and a critical path through requirements, design and Gate 2, protected by a 3-day buffer. The risk register started with 13 risks scored by likelihood and impact and was re-scored at every gate; three were added. The highest exposure was business-rule defects (R10, 16), reduced by specifying the rules as a decision table and testing them to the second. At the end, 10 of 16 risks are closed and the 6 open ones are operational risks that pass to the school (App. B).

## 3 Requirements engineering

Five stakeholders were analysed: the Principal, a lecturer, an HoD, a student representative and the IT officer. Eleven conflicts and ambiguities were resolved in writing; for example, moderation before release is mandatory but releases a whole assignment in one step, and late work is accepted with a penalty of 10% of the maximum mark per day for three days and zero after that, rather than rejected. The SRS follows ISO/IEC/IEEE 29148 with 23 functional and 12 non-functional requirements, each identified, prioritised with MoSCoW and verifiable; every non-functional requirement has a measurable fit criterion, such as “login, including password hashing, completes within 2 seconds”. The interacting rules are stated once as business rules BR-01 to BR-07. The use case model (D3) has 21 use cases for four actors, using include, extend and generalisation, and three are fully dressed: Submit assignment, Mark submission and Moderate and release marks.

## 4 System design

A layered architecture (D4) separates presentation, services, domain, data access and infrastructure; only the data-access layer sees SQL and only the security component sees the password library. Client–server with a database server was rejected because it needs a server and an administrator offline, Model–View–Controller because a

console view and controller are one menu loop, and two layers because SQL and role checks would spread through the menus. The class model of about 55 classes (D5, D6) has an abstract User with four role subclasses and applies four patterns: Strategy for the late penalty, Repository for data access, Factory for creating users, and an injected clock so deadlines can be tested exactly. Observer was rejected because notifications are out of scope, and a State pattern was rejected for the submission life cycle because five states with guarded methods in one class are easier to read and to test. The design follows SOLID; for example, services depend on repository interfaces, not on SQLite. Behaviour is modelled by the submission state machine (D7), an activity diagram with swimlanes from submission to released mark (D8) and sequence diagrams for submitting (D9) and for moderating and releasing (D10). The database (D11) has ten tables in Third Normal Form, with a data dictionary and an SQL schema whose triggers make the mark audit log append-only.

# 5 Implementation

The system has 5,206 lines of production C++17 in 70 files, built with CMake and following a documented coding standard. Each role is a subclass of the abstract User, and the menus call only services. Data is stored in SQLite through repositories using parameterised statements only, and every multi-step write is one transaction. Passwords are stored as salted Argon2id hashes from libsodium, chosen because it is a recognised, memory-hard algorithm with a small C library that builds on Windows and Linux; five failed logins lock an account for 15 minutes. Every mark change is written to the audit log with who, when, the old and new values and the reason. Errors derive from one exception hierarchy and are shown as plain messages, so invalid input or a database error never crashes the program.

# 6 Verification and validation

The test plan maps each V-Model level to the artefact it verifies, with entry and exit criteria, and uses equivalence partitioning and boundary value analysis, for example one second before, on and after a deadline. Catch2 runs 129 tests: unit and integration tests, 18 boundary value tests and 27 system tests derived from the use case flows, including exception flows, driven through the real menus. Business-logic line coverage is 95.2% against a target of 70%. Nine acceptance tests were run and passed, and the defect log records 16 defects with severity, cause, fix and retest result (App. C).

| Measure | Target | Result |
|---|---|---|
| Functional requirements implemented and tested | 23 | 23 of 23 |
| Automated tests passing (Windows and Linux) | All | 129 of 129 |
| Business-logic line coverage (NFR-12) | At least 70% | 95.2% |
| Static analysis findings at release | No high severity | 0 findings |
| Invalid inputs causing a crash (NFR-07) | 0 of 100 | 0 of 100 |
| Acceptance tests | All pass | 9 of 9 |

*Table 1: Main verification results*

# 7 Configuration management and quality assurance

Git follows GitHub Flow: every change is made on a feature branch and merged by pull request into a protected main branch that requires three passing checks. GitHub Actions builds and tests the project on Linux and Windows on every push and runs cppcheck and clang-tidy, which report no findings; the findings fixed and the ones kept on purpose are recorded. Lizard metrics show an average cyclomatic complexity of 2.5, with 90% of the 342 functions in the simple band. A checklist self-review against the class diagrams led to recorded fixes, including the pending-work count of FR-23, which the review found missing (App. F). Releases are tagged with semantic versions: v1.0.0 for the first release and v1.1.0, a minor version, for CR1, because it adds functionality and keeps existing data working.

# 8 Deployment

The UML deployment diagram (D12) shows one Windows school computer running the program as a single process that contains all five layers, with its database, submitted files and daily backups in one data folder, an

external USB disk that receives a copy of every backup, and the GitHub Actions runner that builds each release from a tagged commit, so the school installs exactly the code that was tested. Pushing the tag v1.0.0 made a release workflow build a ready Windows package and publish it with the release notes as a GitHub Release. The installation guide was written for a school technician, and there is a user manual for each role (Appendix D, App. E).

# 9 Maintenance (change request CR1)

CR1 asked for whole-class extensions and for extensions to be shown on the student's list. It is perfective maintenance, because v1.0 did what FR-09 specified and the users asked for new functionality; nothing was faulty and nothing in the environment changed. The impact analysis (App. G) identified every affected requirement, diagram, class, table and test. A new table, class_extension, was chosen over changing the existing extension table or the assignment deadline, because it keeps who, when and why, covers students enrolled later, and leaves existing data untouched; old databases are upgraded automatically on first start. The effective deadline is now the latest of the assignment deadline, class extensions and the student's own extensions. The change was released as v1.1.0 with the class diagrams, state machine and ERD updated in blue. All 115 existing tests passed unchanged, and 14 new tests at every level pass. Phase 1 had predicted this change as risk R06, and keeping extensions in their own entity from Phase 3 meant it needed exactly one new table.

# 10 Professional, legal and ethical issues

**Data protection.** Learners' data is protected by Lesotho's Data Protection Act 2011, and most learners are minors. The system stores only the data needed for assessment, enforces role-based access so learners see only their own records, never leaves the school computer, and keeps a personal extension's reason off the learner's screen. **Security of assessment records.** Marks are hidden until the HoD releases them, the audit log cannot be changed even by the administrator, marking and release are done by different roles, and backups protect against loss. The database file is not encrypted, so the school must keep the computer locked. **Accessibility.** The interface is keyboard-only text of at most 80 columns with no information shown by colour alone, so screen readers can read it, and extensions can serve as an accommodation; it has not yet been tested with disabled users. **Fairness of late penalties.** The rule is applied the same way to everyone, is published, takes a share of the maximum mark rather than the raw mark, and never goes below zero. Its weakness is that one minute late costs as much as a whole day, so the design keeps judgement with people through extensions and audited amendments. Professionally, the work followed the BCS and ACM codes: limitations are stated honestly and only permissively licensed libraries are used.

# 11 Conclusion

Three decisions had the largest effect on the outcome. Specifying the interacting rules once, as business rules with a decision table, turned the highest risk into the best-tested part of the system. Injecting the clock made every deadline boundary testable to the second, which a real clock would not allow. Keeping extensions in their own entity made the change request small. In hindsight, the Waterfall model fitted the fixed brief and the gates, but it delayed feedback from real users until after release; an incremental model with an early prototype shown to lecturers would have tested usability (NFR-06) sooner.

The project delivered a working, tested and documented system that meets all 23 functional requirements, and the maintenance phase showed that the design decisions paid off: the predicted change request needed one new table and no change to the existing tests. The main weaknesses are that the system was tested by scripts rather than by the school's staff and learners, that performance with 800 students was not measured, and that marks already given are not adjusted automatically after a class extension. These are the first steps recommended before the school uses the system for real marks.

# References

ACM (2018) ACM Code of Ethics and Professional Conduct. New York: ACM.

BCS (2022) BCS Code of Conduct. Swindon: BCS, The Chartered Institute for IT.

Cockburn, A. (2001) Writing Effective Use Cases. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) Design Patterns. Reading, MA: Addison-Wesley.
Government of Lesotho (2012) Data Protection Act 2011 (Act No. 5 of 2012). Maseru: Government Printer.
ISO/IEC/IEEE (2018) ISO/IEC/IEEE 29148:2018 Requirements engineering. Geneva: ISO.
Lientz, B.P. and Swanson, E.B. (1980) Software Maintenance Management. Reading, MA: Addison-Wesley.
Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson.